



Smart Trip Management System

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Version 1.0

Prepared for: College Software Engineering Project Submission

Prepared in accordance with IEEE 830 SRS Guidelines

Date: August 2026

Document Control

Field	Details
Project Title	TripEase – Smart Trip Management System
Document Type	Software Requirements Specification (SRS)
Version	1.0
Status	Draft for Faculty Review
Prepared By	TripEase Project Team (Student Developers)
Standard Followed	IEEE 830-1998 SRS Structure (adapted for academic scope)
Intended Use	Requirement analysis, UI/UX design, development, testing, and evaluation

Revision History

Version	Date	Description	Author
0.1	Aug 2026	Initial draft outline created	Project Team
1.0	Aug 2026	Complete SRS finalized for submission	Project Team

Table of Contents

1. Introduction	5
1.1 Purpose	5
1.2 Document Conventions	5
1.3 Intended Audience and Reading Suggestions	5
1.4 Project Scope	5
1.5 Definitions, Acronyms, and Abbreviations	5
1.6 References	6
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Features (Summary)	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	7
2.6 Assumptions and Dependencies	7
2.7 Future Enhancements (Post Phase 1)	7
3. System Features and Functional Requirements	8
3.1 Customer Module	8
3.2 Admin Module	11
4. External Interface Requirements	13
4.1 User Interfaces	13
4.2 Hardware Interfaces	13
4.3 Software Interfaces	13
4.4 Communication Interfaces	13
5. System Architecture and Data Design	14
5.1 High-Level Architecture	14
5.2 Technology Stack	14
5.3 Core Data Entities	14
5.4 Sample Data Schema (JSON)	15
5.5 LocalStorage Key Design	15
5.6 Future Backend Integration Plan	15
6. Non-Functional Requirements	17
7. Other Requirements	19
7.1 Error Handling and Empty States	19

7.2 Data Migration Consideration 19

7.3 Academic/Project Constraints 19

8. Appendices 20

Appendix A: Use Case Summary 20

Appendix B: Glossary (Extended) 20

Appendix C: Assumptions Recap 20

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of **TripEase – Smart Trip Management System**, a frontend-based web application that enables users to plan, organize, and manage complete trips from a single platform. The purpose of this document is to provide a clear, complete, and unambiguous description of the system's intended behaviour so that it can be used as the foundation for design, development, testing, and evaluation of the project by students and faculty.

1.2 Document Conventions

Functional requirements in this document are identified using the prefix **FR-XX**, non-functional requirements use the prefix **NFR-XX**, and use cases use the prefix **UC-XX**. Priority levels are classified as **High** (must-have for core functionality), **Medium** (important but not blocking), and **Low** (enhancement / future scope). Headings follow a numbered hierarchy consistent with IEEE 830 conventions.

1.3 Intended Audience and Reading Suggestions

This document is intended for the student development team, project guide/faculty evaluators, UI/UX designers, and reviewers assessing the project for academic submission. Developers should focus on Sections 3–5 for functional and architectural detail; evaluators may focus on Sections 1–2 and 6 for scope and quality expectations; designers should refer to Section 4 for interface requirements.

1.4 Project Scope

TripEase is a responsive, green travel-themed web application focused on **trip planning and itinerary management** rather than actual travel booking or payment processing. The initial version (Phase 1) is a frontend-only implementation using mock JSON data and browser LocalStorage to simulate persistence. The system is architected so that a real backend (REST APIs, database, authentication service, maps, weather, and currency conversion APIs) can be integrated in a later phase without redesigning the frontend. The project consists of two modules:

- **Customer Module** — destination discovery, trip creation, day-wise itinerary building, budget planning, packing checklist, favorites, and a personal trip dashboard.
- **Admin Module** — management of destinations, places, activities, featured content, and basic platform statistics through an administrative dashboard.

Out of scope for this version: real payment gateways, live hotel/flight booking, real-time GPS navigation, and multi-language support. These are listed as future enhancements in Section 2.7.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
CRUD	Create, Read, Update, Delete
Itinerary	A day-wise structured plan of activities for a trip
Mock Data	Static/sample JSON data used to simulate a backend during Phase 1
LocalStorage	Browser-based key-value storage used for client-side persistence

Term	Definition
REST API	Representational State Transfer Application Programming Interface
Admin	A privileged user who manages platform content
Customer / Traveler	An end user who plans and manages personal trips
Responsive Design	UI layout that adapts to desktop, tablet, and mobile screens

1.6 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Nielsen Norman Group — Usability Heuristics for User Interface Design.
- Material Design and modern travel-platform UI patterns (used as visual inspiration only).
- MDN Web Docs — Web Storage API (LocalStorage) reference.

2. Overall Description

2.1 Product Perspective

TripEase is a new, self-contained frontend product. It is not a modification of an existing system. Phase 1 operates entirely on the client side, using mock JSON datasets bundled with the application and browser LocalStorage for persisting user-created data such as trips, itineraries, budgets, and favorites. The architecture uses a clear service-layer abstraction so that mock-data services can later be swapped with real REST API calls, allowing the system to evolve into a full client-server application without major rework of UI components.

2.2 Product Features (Summary)

- Destination discovery with search, filters, and category browsing.
- Detailed destination pages with images, description, best time to visit, and highlights.
- Guided trip creation (destination, dates, travelers, budget, interests).
- Day-wise interactive itinerary builder with add/edit/delete/reorder of activities.
- Smart budget planner with category-wise estimated vs. planned expense tracking.
- Packing checklist with default templates and custom items.
- Favorites / wishlist for destinations.
- Personal dashboard listing all trips with status (Upcoming, Ongoing, Completed).
- Admin dashboard to manage destinations, places, activities, and featured content.
- Platform statistics overview for admins (counts of destinations, trips, users — mock-derived).
- Notifications, loading states, and meaningful empty/error states throughout the UI.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Customer / Traveler	Plans trips, builds itineraries, tracks budget and packing.	Basic — general web/app user
Administrator	Manages destinations, places, activities, and views statistics.	Moderate — comfortable with admin panels
Guest Visitor	Browses destinations without an account (limited actions).	Basic

2.4 Operating Environment

- Runs in modern web browsers: Chrome, Firefox, Edge, Safari (latest two versions).
- Responsive layout supporting desktop ($\geq 1200\text{px}$), tablet ($\geq 768\text{px}$), and mobile ($\geq 320\text{px}$) viewports.
- Client-side only in Phase 1 — no server/database required to run the application.
- Built using standard web technologies (see Section 5.2 Technology Stack).

2.5 Design and Implementation Constraints

- Phase 1 must not depend on a live backend; all data is served from local mock JSON files.
- Persistence is limited to browser LocalStorage, which is per-browser and per-device.
- The UI must consistently follow a green, travel-inspired visual theme.
- The codebase must be modular so that a data-service layer can later be pointed at real APIs with minimal changes to components.
- Project timeline and team size are constrained to a typical college semester project scope; features are intentionally kept realistic and achievable.

2.6 Assumptions and Dependencies

- Users have a device with a modern browser and internet access to load the application and its assets.
- Mock destination data is sufficiently representative for demonstration and evaluation purposes.
- LocalStorage is available and not disabled/cleared by the user during a session or between demos.
- Actual payment, booking, and third-party travel APIs are assumed unavailable in this phase and are simulated or omitted.

2.7 Future Enhancements (Post Phase 1)

- Integration with a real backend and relational/NoSQL database (Node.js/Express + MongoDB or similar), replacing mock JSON with REST APIs.
- Real user authentication and authorization (JWT-based) replacing mock/local auth.
- Integration with Maps API (e.g., Google Maps) for destination location and route visualization.
- Integration with Weather API for destination-based forecasts within the itinerary.
- Currency conversion API for multi-currency budget planning.
- Real booking/payment gateway integration for flights, hotels, and activities.
- Push/email notifications and collaborative trip planning (shared itineraries).

3. System Features and Functional Requirements

This section details the functional requirements of TripEase, organized by module and feature. Each feature includes a description, priority, and a set of testable functional requirements.

3.1 Customer Module

3.1.1 User Registration and Authentication (Mock)

Description: Allows a customer to create an account and log in. In Phase 1, authentication is simulated using mock/local validation and LocalStorage session data.

Priority: High

ID	Requirement
FR-01	The system shall allow a new user to register with name, email, and password.
FR-02	The system shall allow a registered user to log in and log out.
FR-03	The system shall validate email format and required fields before submission.
FR-04	The system shall persist the logged-in session using LocalStorage until logout.
FR-05	The system shall display appropriate error messages for invalid login attempts.

3.1.2 Destination Discovery, Search, and Filter

Description: Enables users to browse destinations and narrow results using search text and filters (category, budget range, popularity, region).

Priority: High

ID	Requirement
FR-06	The system shall display destinations as attractive cards with image, name, region, and rating.
FR-07	The system shall allow users to search destinations by name or keyword.
FR-08	The system shall allow filtering by category (e.g., beach, hill station, heritage, adventure).
FR-09	The system shall allow filtering by estimated budget range.
FR-10	The system shall display an empty state when no destinations match the applied filters.

3.1.3 Destination Details View

Description: Displays complete information about a selected destination, including gallery, description, highlights, best time to visit, and popular activities.

Priority: High

ID	Requirement
FR-11	The system shall display a detailed page for each destination on selection.
FR-12	The system shall display an image gallery, description, and key highlights.
FR-13	The system shall show suggested activities/places associated with the destination.
FR-14	The system shall provide a call-to-action to start planning a trip to that destination.

3.1.4 Trip Creation (Guided Setup)

Description: Allows a user to create a new trip by specifying destination, travel dates, number of travelers, budget, and interests, forming the base for the itinerary.

Priority: High

ID	Requirement
FR-15	The system shall allow the user to select a destination for a new trip.
FR-16	The system shall allow the user to select start and end travel dates.
FR-17	The system shall allow the user to specify number of travelers.
FR-18	The system shall allow the user to set a total estimated budget.
FR-19	The system shall allow the user to select interests/tags (e.g., nature, food, culture).
FR-20	The system shall validate that the end date is not earlier than the start date.
FR-21	The system shall save the newly created trip and redirect the user to the itinerary builder.

3.1.5 Day-wise Interactive Itinerary Builder

Description: Core planning feature allowing users to organize their trip into a day-by-day timeline of activities.

Priority: High

ID	Requirement
FR-22	The system shall automatically generate day slots based on the trip's date range.
FR-23	The system shall allow users to add an activity to a specific day with name, time, location, and notes.
FR-24	The system shall allow users to edit an existing activity.
FR-25	The system shall allow users to delete an activity from the itinerary.
FR-26	The system shall allow users to reorder activities within a day via drag-and-drop or up/down controls.
FR-27	The system shall visually present the itinerary as an interactive timeline per day.
FR-28	The system shall persist itinerary changes automatically to LocalStorage.

3.1.6 Smart Budget Planner

Description: Helps users track estimated costs against their trip budget across categories such as travel, stay, food, and activities.

Priority: High

ID	Requirement
FR-29	The system shall allow users to define budget categories (e.g., travel, accommodation, food, activities, shopping).
FR-30	The system shall allow users to add estimated expenses under each category.
FR-31	The system shall calculate and display total planned expense versus total budget.
FR-32	The system shall display a visual breakdown (e.g., progress bar or chart) of budget utilization.
FR-33	The system shall alert the user when planned expenses exceed the set budget.

3.1.7 Packing Checklist

Description: Provides a simple, trip-specific checklist to help users track packing progress.

Priority: Medium

ID	Requirement
FR-34	The system shall provide a default packing checklist template based on trip type.
FR-35	The system shall allow users to add, edit, and remove custom checklist items.
FR-36	The system shall allow users to mark items as packed/unpacked.
FR-37	The system shall display packing progress (e.g., "8 of 12 items packed").

3.1.8 Favorites / Wishlist

Description: Allows users to bookmark destinations they are interested in for future planning.

Priority: Medium

ID	Requirement
FR-38	The system shall allow users to mark or unmark a destination as favorite.
FR-39	The system shall display a dedicated favorites list on the user's dashboard.
FR-40	The system shall persist favorites in LocalStorage per user.

3.1.9 Personal Trip Dashboard

Description: Central place for a user to view and manage all their trips.

Priority: High

ID	Requirement
FR-41	The system shall display all trips created by the logged-in user.
FR-42	The system shall categorize trips as Upcoming, Ongoing, or Completed based on dates.
FR-43	The system shall allow users to open, edit, or delete an existing trip.
FR-44	The system shall show a summary card per trip (destination, dates, budget status, itinerary completion).

3.1.10 Notifications and System Feedback

Description: Provides users with contextual feedback for actions and system states.

Priority: Medium

ID	Requirement
FR-45	The system shall show toast/notification messages for successful actions (e.g., "Trip saved").
FR-46	The system shall display loading indicators during data-fetch simulations.
FR-47	The system shall display meaningful empty states (e.g., "No trips yet — start planning!").
FR-48	The system shall display clear error states for failed actions or invalid input.

3.2 Admin Module

3.2.1 Admin Authentication

Description: Provides a separate, mock-secured login for administrators.

Priority: High

ID	Requirement
FR-49	The system shall provide a distinct admin login screen.
FR-50	The system shall restrict access to admin routes to authenticated admin sessions only.

3.2.2 Manage Destinations

Description: Allows admins to maintain the destination catalog shown to customers.

Priority: High

ID	Requirement
FR-51	The system shall allow the admin to add a new destination with name, region, category, images, description, and price indicator.
FR-52	The system shall allow the admin to edit existing destination details.
FR-53	The system shall allow the admin to delete a destination.
FR-54	The system shall list all destinations in a searchable, tabular admin view.

3.2.3 Manage Places and Activities

Description: Allows admins to maintain the pool of places/activities that customers can add to their itineraries.

Priority: High

ID	Requirement
FR-55	The system shall allow the admin to add, edit, and delete places/activities linked to a destination.
FR-56	The system shall allow the admin to categorize activities (e.g., sightseeing, food, adventure).

3.2.4 Manage Featured Destinations

Description: Allows admins to curate which destinations are highlighted on the homepage.

Priority: Medium

ID	Requirement
FR-57	The system shall allow the admin to mark or unmark a destination as "Featured".
FR-58	The system shall display featured destinations prominently on the customer homepage.

3.2.5 Platform Statistics Dashboard

Description: Gives admins a quick overview of platform activity derived from mock/local data.

Priority: Medium

ID	Requirement
FR-59	The system shall display total counts of destinations, places, and registered users.
FR-60	The system shall display the number of trips created across the platform.
FR-61	The system shall present basic statistics using summary cards and simple charts.

4. External Interface Requirements

4.1 User Interfaces

The application shall use a modern, green, travel-inspired visual theme throughout, evoking nature and journeys while maintaining strong readability and contrast. Key screens include:

- Landing / Home page with hero banner, search bar, and featured destinations.
- Destination listing page with search and filter panel.
- Destination detail page with gallery and highlights.
- Trip creation wizard (multi-step form).
- Itinerary builder with day tabs and drag-and-drop activity cards.
- Budget planner with category breakdown and progress visualizations.
- Packing checklist page.
- Customer dashboard (My Trips, Favorites, Profile).
- Admin dashboard with statistics cards and management tables for destinations/places/activities.

All screens shall be responsive and shall gracefully adapt layout, navigation (e.g., collapsing to a hamburger menu on mobile), and card grids across desktop, tablet, and mobile breakpoints. The UI shall include loading skeletons/spinners, toast notifications, and empty/error state illustrations with guidance text.

4.2 Hardware Interfaces

No special hardware is required. The system runs on any standard desktop, laptop, tablet, or smartphone capable of running a modern web browser.

4.3 Software Interfaces

Interface	Description
Browser LocalStorage	Used in Phase 1 to persist trips, itineraries, budgets, checklists, and favorites on the client.
Mock JSON Data Files	Bundled static data simulating destinations, places, activities, and users for Phase 1.
Future REST API Layer	Planned interface for CRUD operations against a real backend (see Section 5.6).
Future Maps / Weather / Currency APIs	Planned third-party integrations for enriched trip-planning features.

4.4 Communication Interfaces

Phase 1 requires only standard HTTPS delivery of the static web application and its assets. When the future REST API layer is introduced, communication will occur over HTTPS using JSON payloads, following standard request/response conventions.

5. System Architecture and Data Design

5.1 High-Level Architecture

TripEase follows a layered frontend architecture designed for a clean future transition to a client-server model:

- **Presentation Layer** — Reusable UI components (cards, forms, timelines, charts, modals) organized by feature (destinations, trips, itinerary, budget, admin).
- **State Management Layer** — Manages application state such as current user, active trip, and UI state (loading/error/notifications).
- **Data Service Layer** — An abstraction (e.g., destinationService, tripService, budgetService) that currently reads/writes mock JSON and LocalStorage, but exposes the same function signatures a future API-backed service would use.
- **Mock Data / LocalStorage Layer (Phase 1)** — Static JSON files for catalog data (destinations, places, activities) and LocalStorage for user-generated data (trips, itineraries, budgets, checklists, favorites, session).
- **Future Backend Layer (Phase 2+)** — REST API + database replacing the mock layer behind the same service-layer interface.

5.2 Technology Stack

Layer	Suggested Technology
Frontend Framework	React.js (component-based, widely used for academic projects)
Styling	Tailwind CSS / CSS Modules for the green travel theme
State Management	React Context API or a lightweight state library
Client-side Persistence	Browser LocalStorage (Web Storage API)
Mock Data	Static JSON files bundled with the frontend
Charts / Visualizations	A lightweight charting library for budget visualizations
Future Backend (Phase 2)	Node.js + Express, REST APIs
Future Database (Phase 2)	MongoDB or MySQL/PostgreSQL
Future Integrations	Maps API, Weather API, Currency Conversion API

5.3 Core Data Entities

Entity	Key Attributes
User	id, name, email, password (mock-hashed), role (customer/admin)
Destination	id, name, region, category, images[], description, avgCost, rating, featured
Place / Activity	id, destinationId, name, category, description, estimatedCost, duration
Trip	id, userId, destinationId, startDate, endDate, travelers, budget, interests[], status
ItineraryDay	id, tripId, dayNumber, date, activities[]
ItineraryActivity	id, dayId, title, time, location, notes, order
BudgetItem	id, tripId, category, estimatedCost, plannedCost
PackingItem	id, tripId, name, isPacked, isCustom

Entity	Key Attributes
Favorite	id, userId, destinationId

5.4 Sample Data Schema (JSON)

Example structure for a Trip object as stored in Phase 1:

```
{
  "id": "trip_1001",
  "userId": "user_204",
  "destinationId": "dest_012",
  "startDate": "2026-11-10",
  "endDate": "2026-11-14",
  "travelers": 2,
  "budget": 25000,
  "interests": ["nature", "food"],
  "status": "Upcoming",
  "itinerary": [
    { "day": 1, "date": "2026-11-10",
      "activities": [
        { "id": "act_1", "title": "Arrival & Check-in", "time": "14:00", "order": 1 }
      ]
    }
  ]
}
```

Caption: Sample Trip object showing nested itinerary structure.

5.5 LocalStorage Key Design

Key Pattern	Purpose
tripease_session	Stores the currently logged-in user's session info
tripease_users	Stores mock-registered user accounts
tripease_trips_{userId}	Stores all trips created by a given user
tripease_favorites_{userId}	Stores a user's favorite destinations
tripease_admin_destinations	Stores admin-managed destination catalog overrides

5.6 Future Backend Integration Plan

The data-service layer is designed so each function (e.g., getDestinations(), createTrip()) can be re-pointed from a mock/LocalStorage implementation to an HTTP call with the same signature. Planned REST endpoints for Phase 2 include:

Method	Endpoint	Purpose
GET	/api/destinations	Fetch destination catalog with filters
GET	/api/destinations/:id	Fetch a single destination's details
POST	/api/trips	Create a new trip
GET	/api/trips/:userId	Fetch all trips for a user
PUT	/api/trips/:id/itinerary	Update a trip's itinerary
PUT	/api/trips/:id/budget	Update a trip's budget items
POST	/api/admin/destinations	Admin: create a destination

Method	Endpoint	Purpose
PUT/DELETE	/api/admin/destinations/:id	Admin: update/delete a destination

6. Non-Functional Requirements

6.1 Performance

ID	Requirement
NFR-01	Destination listing pages shall load and render within 2 seconds under normal conditions using mock data.
NFR-02	Itinerary drag-and-drop reordering shall reflect UI changes within 300 ms.

6.2 Usability

ID	Requirement
NFR-03	The interface shall follow a consistent green travel theme with clear visual hierarchy and legible typography.
NFR-04	Core tasks (create trip, add activity, set budget) shall be completable within 3 clicks/taps from the relevant screen.
NFR-05	The system shall provide clear feedback (notifications, states) for every user action.

6.3 Reliability and Availability

ID	Requirement
NFR-06	User-entered data (trips, itineraries, budgets) shall not be lost during normal browser use, as it is persisted to LocalStorage on every change.
NFR-07	The application shall handle missing or malformed mock data gracefully without crashing, showing an error state instead.

6.4 Security

ID	Requirement
NFR-08	Passwords, even in mock form, shall not be stored or displayed in plain readable form in the UI.
NFR-09	Admin routes/pages shall not be accessible without a valid admin session.

6.5 Maintainability

ID	Requirement
NFR-10	The codebase shall be organized into reusable, feature-based components and a clearly separated data-service layer.
NFR-11	Mock data structures shall mirror the planned future API response shapes to minimize rework during backend integration.

6.6 Portability and Responsiveness

ID	Requirement
NFR-12	The application shall function correctly on the latest two versions of major browsers (Chrome, Firefox, Edge, Safari).
NFR-13	The layout shall adapt responsively across desktop, tablet, and mobile screen sizes.

6.7 Scalability (Design-Level)

ID	Requirement
NFR-14	The architecture shall allow the mock data layer to be replaced by a real API without redesigning UI components, supporting future growth in data volume and users.

7. Other Requirements

7.1 Error Handling and Empty States

Every list-based or data-driven screen (destinations, trips, itinerary, budget, favorites, admin tables) shall define a distinct empty state (no data yet) and error state (failed to load/save), each with a short, friendly message and, where applicable, a call-to-action (e.g., "Plan your first trip").

7.2 Data Migration Consideration

Because Phase 1 data lives in LocalStorage, the design shall keep data shapes close to the future API contract so that a migration utility can later import LocalStorage data into the real backend database with minimal transformation.

7.3 Academic/Project Constraints

The feature set defined in this document is intentionally scoped to be achievable by a small student team within a single semester, while remaining representative of a professional, real-world travel-planning product for demonstration and evaluation purposes.

8. Appendices

Appendix A: Use Case Summary

Use Case ID	Use Case	Actor
UC-01	Register / Log in	Customer
UC-02	Search and filter destinations	Customer
UC-03	View destination details	Customer / Guest
UC-04	Create a new trip	Customer
UC-05	Build and edit day-wise itinerary	Customer
UC-06	Track budget against plan	Customer
UC-07	Maintain packing checklist	Customer
UC-08	Manage favorite destinations	Customer
UC-09	View personal trip dashboard	Customer
UC-10	Manage destinations, places, and activities	Admin
UC-11	Manage featured destinations	Admin
UC-12	View platform statistics	Admin

Appendix B: Glossary (Extended)

Term	Meaning
Trip Status	Lifecycle state of a trip: Upcoming, Ongoing, or Completed, derived from dates
Featured Destination	A destination curated by an admin for homepage promotion
Empty State	UI shown when a list/section has no data yet
Service Layer	Abstraction layer isolating UI components from the underlying data source

Appendix C: Assumptions Recap

- This SRS covers Phase 1 (frontend-only, mock data) unless explicitly noted as a future enhancement.
- All monetary values are treated as a single default currency in Phase 1.
- The document will be revised as the design evolves during development and review.

End of Document — TripEase Software Requirements Specification, Version 1.0